

Universidad ORT Uruguay
Facultad de Ingeniería
Escuela de Tecnología

OBLIGATORIO PROGRAMACION 1
DOCUMENTO DE ANÁLISIS



Nazarena Barrios – 376371



Gerard Figueroa – 268897

Grupo: N1B

Docente: Danis Carlos Chabiano Jiménez

Analista en tecnologías de la información (25-06-2026)

Índice

1.	Descripción general del problema a resolver	5
1.1.	Tipos de usuario del sistema.....	5
1.2.	Listado de funcionalidades	5
2.	Detalle de Funcionalidades.....	6
2.1	F01 – Registro de postulante.	6
2.1.1	Acceso	6
2.1.2	Descripción.....	6
2.1.3	Interfaz de usuario	6
2.1.4	Validaciones	6
2.2	F02 – Inicio sesión.....	7
2.2.1	Acceso	7
2.2.2	Descripción.....	7
2.2.3	Interfaz de usuario	7
2.2.4	Validaciones	7
2.3	F03 – Cierre sesión.....	8
2.3.1	Acceso	8
2.3.2	Descripción.....	8
2.3.3	Interfaz de usuario	8
2.3.4	Validaciones	8
2.4	F04 – Visualizar ofertas laborales activas	8
2.4.1	Acceso	8
2.4.2	Descripción.....	8
2.4.3	Interfaz de usuario	9

2.4.4	Validaciones	9
2.5	F05 – Postularse a una oferta laboral	10
2.5.1	Acceso	10
2.5.2	Descripción.....	10
2.5.3	Interfaz de usuario	10
2.5.4	Validaciones	10
2.6	F06 – Ver mis postulaciones.	10
2.6.1	Acceso	10
2.6.2	Descripción.....	10
2.6.3	Interfaz de usuario	11
2.6.4	Validaciones	11
2.7	F07 – Visualizar ofertas destacadas.	11
2.7.1	Acceso	11
2.7.2	Descripción.....	11
2.7.3	Interfaz de usuario	12
2.7.4	Validaciones	12
2.8	F08 – Crear oferta laboral.....	12
2.8.1	Acceso	12
2.8.2	Descripción.....	12
2.8.3	Interfaz de usuario	13
2.8.4	Validaciones	13
2.9	F09 – Editar oferta laboral.....	13
2.9.1	Acceso	13
2.9.2	Descripción.....	13
2.9.3	Interfaz de usuario	14
2.9.4	Validaciones	14

2.10	F10 – Cerrar oferta laboral.	14
2.10.1	Acceso	14
2.10.2	Descripción.....	15
2.10.3	Interfaz de usuario	15
2.10.4	Validaciones	15
2.11	F11 – Visualizar listado de ofertas.	15
2.11.1	Acceso	15
2.11.2	Descripción.....	15
2.11.3	Interfaz de usuario	16
2.11.4	Validaciones	16
2.12	F12 – Procesar postulaciones.	16
2.12.1	Acceso	16
2.12.2	Descripción.....	16
2.12.3	Interfaz de usuario	17
2.12.4	Validaciones	17
2.13	F13 – Visualizar estadísticas.	17
2.13.1	Acceso	17
2.13.2	Descripción.....	18
2.13.3	Interfaz de usuario	18
2.13.4	Validaciones	18
3.	Casos de prueba.	19
1.3.	Contexto de uso.	22

1. Descripción general del problema a resolver

Se desea desarrollar una aplicación web para una cartelera de postulaciones laborales que permita conectar postulantes con empresas que publiquen ofertas de trabajo.

1.1. Tipos de usuario del sistema

El sistema contará con dos tipos de usuarios:

- **Postulante:** usuario que podrá registrarse en la aplicación, iniciar sesión, visualizar ofertas laborales compatibles con su perfil y realizar postulaciones.
- **Administrador:** usuario encargado de gestionar las ofertas laborales, procesar postulaciones y visualizar estadísticas generales del sistema.

1.2. Listado de funcionalidades

- [F01 – Registro de postulante – Usuario: Postulante
- F02 – Inicio de sesión – Usuario: Postulante y Administrador
- F03 – Cierre de sesión – Usuario: Postulante y Administrador
- F04 – Visualizar ofertas laborales activas – Usuario: Postulante
- F05 – Postularse a una oferta laboral – Usuario: Postulante
- F06 – Ver mis postulaciones – Usuario: Postulante
- F07 – Visualizar ofertas destacadas – Usuario: Postulante
- F08 – Crear oferta laboral – Usuario: Administrador
- F09 – Editar oferta laboral – Usuario: Administrador
- F10 – Cerrar oferta laboral – Usuario: Administrador
- F11 – Visualizar listado de ofertas – Usuario: Administrador
- F12 – Procesar postulaciones – Usuario: Administrador

- F13 – Visualizar estadísticas – Usuario: Administrador

2. Detalle de Funcionalidades

A continuación, se presenta el detalle de cada una de las funcionalidades a resolver en el obligatorio. Las funcionalidades están ordenadas por tipo de usuario.

2.1 F01 – Registro de postulante.

2.1.1 Acceso

Puede acceder el postulante.

2.1.2 Descripción

El postulante deberá registrarse en la aplicación ingresando un nombre de usuario, contraseña, nombre completo, nivel de experiencia y área de interés.

Todos los datos serán obligatorios y el sistema verificará que el usuario no exista previamente y que la contraseña cumpla con los requisitos establecidos.

2.1.3 Interfaz de usuario

2.1.4 Validaciones

- Todos los campos son obligatorios.
- El nombre de usuario debe tener mínimo 5 caracteres.
- El nombre de usuario no puede existir previamente en el sistema.
- La validación del usuario será case insensitive.

- La contraseña debe contener mínimo 5 caracteres.
- La contraseña debe incluir al menos:
 - una mayúscula,
 - una minúscula,
 - un número.
 - En caso de error el sistema mostrará el mensaje correspondiente.

2.2F02 – Inicio sesión

2.2.1 Acceso

Puede acceder el postulantes y administradores.

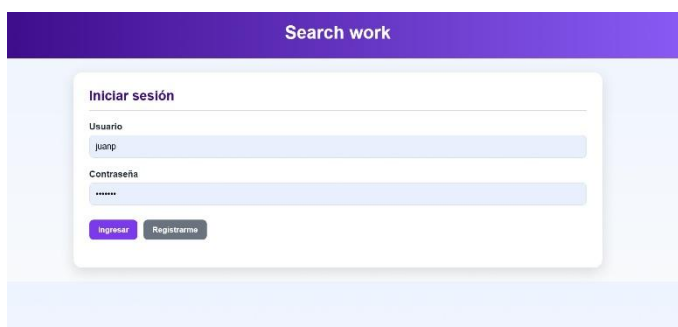
2.2.2 Descripción

Los usuarios podrán ingresar a la aplicación utilizando su nombre de usuario y contraseña.

El sistema verificará que las credenciales ingresadas sean correctas y permitirá acceder a las funcionalidades correspondientes según el tipo de usuario.

El registro de los administradores no será necesario ya que forma parte de las precargas

2.2.3 Interfaz de usuario



The screenshot displays a web application interface. At the top, there is a purple header bar with the text "Search work". Below this, a white login form is centered. The form has a title "Iniciar sesión". It contains two input fields: "Usuario" with the value "juanp" and "Contraseña" with masked characters. Below the fields are two buttons: a purple "Ingresar" button and a grey "Regístrate" button.

2.2.4 Validaciones

- Todos los campos son obligatorios.
- El usuario debe existir en el sistema.

- La contraseña debe coincidir con la registrada para el usuario.
- La contraseña será case sensitive.
- En caso de datos incorrectos el sistema mostrará un mensaje de error correspondiente.

2.3F03 – Cierre sesión

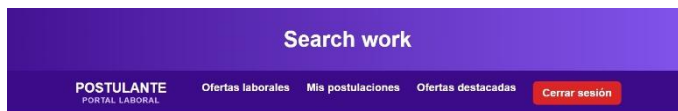
2.3.1 Acceso

Puede acceder el postulantes y administradores.

2.3.2 Descripción

Los usuarios podrán cerrar la sesión activa en cualquier momento para volver a la pantalla de inicio de sesión o permitir el ingreso de otro usuario al sistema.

2.3.3 Interfaz de usuario



2.3.4 Validaciones

- El sistema deberá finalizar correctamente la sesión activa.
- Luego de cerrar sesión el usuario no podrá acceder a funcionalidades privadas sin volver a iniciar sesión.

2.4F04 – Visualizar ofertas laborales activas

2.4.1 Acceso

Puede acceder el postulante.

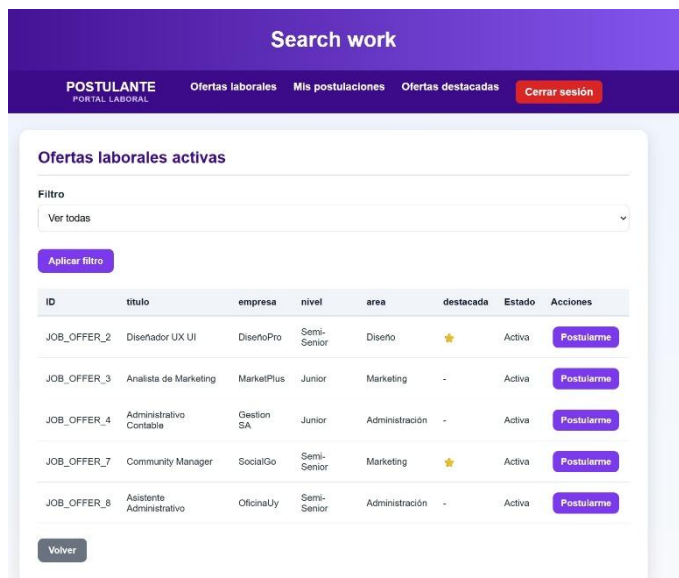
2.4.2 Descripción

El postulante podrá visualizar un listado de ofertas laborales activas compatibles con su perfil.

Por defecto, el sistema mostrará únicamente las ofertas correspondientes a su área de interés, aunque el usuario podrá ampliar el listado para visualizar todas las ofertas disponibles.

Solo se mostrarán ofertas activas y a las cuales el postulante pueda postularse, en caso de querer ver todas las ofertas se mostraran las activas pero en las que el postulante no se pueda postular el botón estará deshabilitado .

2.4.3 Interfaz de usuario



2.4.4 Validaciones

- Solo deberán mostrarse ofertas activas.
- El postulante Junior no podrá visualizar ofertas Senior.
- El postulante Senior no podrá visualizar ofertas Junior.
- Los postulantes Semi-Senior podrán visualizar ofertas de cualquier nivel.
- No deberán mostrarse ofertas a las que el usuario ya se haya postulado previamente.
- El sistema deberá mostrar únicamente ofertas disponibles para postularse.

2.5F05 – Postularse a una oferta laboral

2.5.1 Acceso

Puede acceder el postulante.

2.5.2 Descripción

El postulante podrá realizar una postulación a una oferta laboral disponible.

Al confirmar la postulación, el sistema creará una nueva postulación con un identificador único autoincremental y estado inicial “Pendiente”.

2.5.3 Interfaz de usuario

ID	título	empresa	nivel	area	destacada	Estado	Acciones
JOB_OFFER_2	Diseñador UX UI	DiseñoPro	Semi-Senior	Diseño	★	Activa	Postularme

2.5.4 Validaciones

- La oferta debe encontrarse activa.
- El postulante debe cumplir con el nivel de experiencia requerido.
- El postulante no puede haberse postulado previamente a la misma oferta.
- La postulación deberá generarse con un identificador único con formato “JOB_nroAutoincremental”.
- El estado inicial de la postulación será “Pendiente”.
- En caso de error el sistema mostrará el mensaje correspondiente.

2.6F06 – Ver mis postulaciones.

2.6.1 Acceso

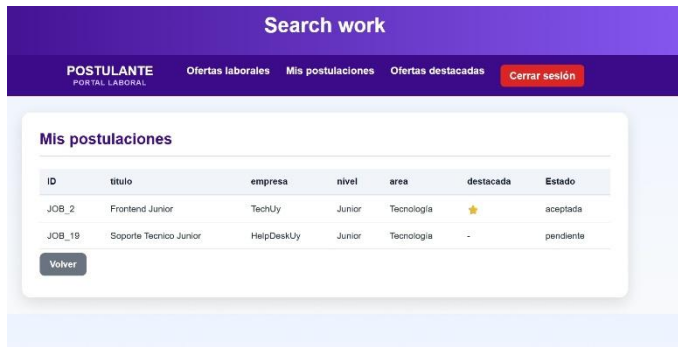
Puede acceder el postulante.

2.6.2 Descripción

El postulante podrá visualizar un listado con todas las postulaciones realizadas en el sistema.

El listado mostrará el nombre de la oferta y el estado actual de cada postulación, pudiendo ser “Pendiente”, “Aceptada” o “Rechazada”.

2.6.3 Interfaz de usuario



2.6.4 Validaciones

- La oferta debe encontrarse activa.
- El sistema deberá mostrar únicamente las postulaciones correspondientes al usuario logueado.
- El estado mostrado deberá coincidir con el estado actual de cada postulación.
- El listado deberá incluir postulaciones pendientes, aceptadas y rechazadas.

2.7F07 – Visualizar ofertas destacadas.

2.7.1 Acceso

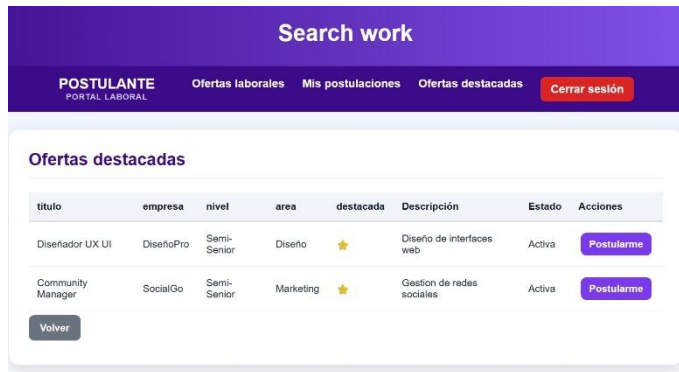
Puede acceder el postulante.

2.7.2 Descripción

El postulante podrá visualizar un catálogo de ofertas laborales destacadas marcadas previamente por el administrador.

El catálogo mostrará información básica de cada oferta y permitirá realizar postulaciones desde la misma interfaz.

2.7.3 Interfaz de usuario



2.7.4 Validaciones

- Solo deberán mostrarse ofertas marcadas como destacadas.
- Solo deberán visualizarse ofertas activas.
- El postulante únicamente podrá visualizar ofertas compatibles con su nivel de experiencia.
- No deberán mostrarse ofertas a las que el usuario ya se haya postulado previamente.
- El sistema deberá permitir postularse desde esta interfaz.

2.8F08 – Crear oferta laboral.

2.8.1 Acceso

Puede acceder el administrador

2.8.2 Descripción

El administrador podrá crear nuevas ofertas laborales ingresando toda la información correspondiente a la oferta.

Cada oferta creada tendrá un identificador único autoincremental con formato “JOB_OFFER_nroAutoincremental” y estado inicial “Activa”.

2.8.3 Interfaz de usuario

Search work

ADMIN
PANEL DE CONTROL

Crear oferta laboral Listado de ofertas Postulaciones pendientes Estadísticas Cerrar sesión

Crear oferta laboral

Título de la oferta

Empresa

Descripción

Nivel requerido
Seleccione...

Área
Seleccione...

Limite de postulaciones

Cantidad de vacantes

Oferta destacada
Seleccione...

Guardar oferta Volver

2.8.4 Validaciones

- Todos los campos son obligatorios excepto “Oferta destacada”.
- El límite de postulaciones debe ser mayor o igual a la cantidad de vacantes.
- Los valores numéricos deberán ser mayores a 0.
- La oferta deberá generarse con un identificador único autoincremental.
- El estado inicial de la oferta será “Activa”.
- En caso de error el sistema mostrará el mensaje correspondiente.

2.9F09 – Editar oferta laboral.

2.9.1 Acceso

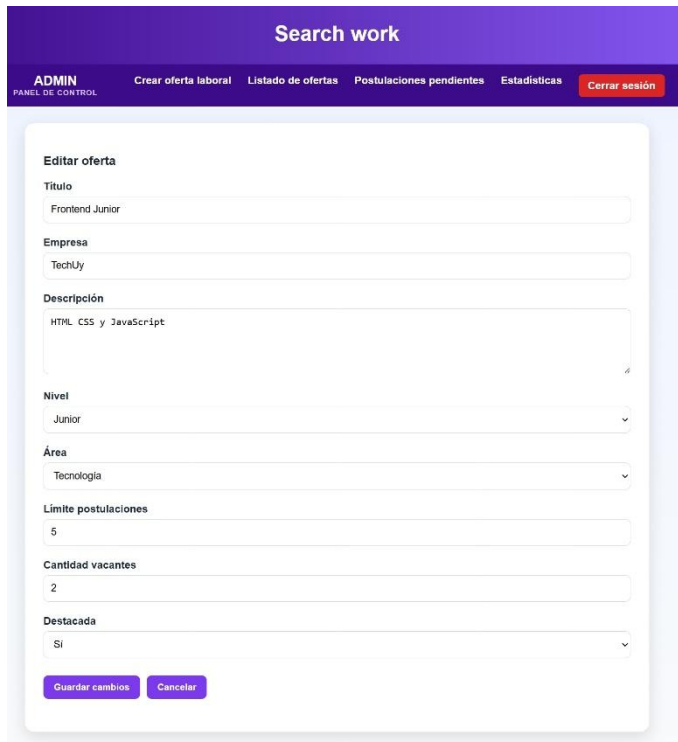
Puede acceder el administrador.

2.9.2 Descripción

El administrador podrá modificar la información de una oferta laboral existente del sistema.

Podrá editar datos como el título, empresa, descripción, nivel requerido, área, límite de postulaciones, cantidad de vacantes y si la oferta es destacada o no.

2.9.3 Interfaz de usuario



The screenshot displays the 'Search work' admin interface. At the top, there is a purple header with the title 'Search work' and a navigation bar containing links: 'ADMIN', 'Crear oferta laboral', 'Listado de ofertas', 'Postulaciones pendientes', 'Estadísticas', and a 'Cerrar sesión' button. Below the header, the main content area is titled 'Editar oferta'. It contains several form fields: 'Titulo' (Frontend Junior), 'Empresa' (TechUy), 'Descripción' (HTML, CSS y JavaScript), 'Nivel' (Junior), 'Área' (Tecnología), 'Límite postulaciones' (5), 'Cantidad vacantes' (2), and 'Destacada' (Si). At the bottom of the form, there are two buttons: 'Guardar cambios' and 'Cancelar'.

2.9.4 Validaciones

- Todos los campos obligatorios deberán contener datos válidos.
- El límite de postulaciones deberá ser mayor o igual a la cantidad de vacantes.
- Los valores numéricos deberán ser mayores a 0.
- La modificación deberá mantenerse sobre la misma oferta seleccionada.
- El sistema deberá actualizar correctamente la información modificada.
- En caso de error el sistema mostrará el mensaje correspondiente.

2.10 F10 – Cerrar oferta laboral.

2.10.1 Acceso

Puede acceder el administrador.

2.10.2 Descripción

El administrador podrá cerrar una oferta laboral existente.

Esta acción no elimina la oferta del sistema, sino que cambia su estado a “Cerrada”, realizando una eliminación lógica.

2.10.3 Interfaz de usuario

ID	título	empresa	nivel	area	destacada	Estado	Acciones
JOB_OFFER_1	Frontend Junior	TechUy	Junior	Tecnología	si	Activa	Editar Cerrar

2.10.4 Validaciones

- La oferta debe existir en el sistema.
- La oferta no deberá eliminarse definitivamente.
- El estado de la oferta deberá cambiar a “Cerrada”.
- Las ofertas cerradas no deberán aparecer disponibles para los postulantes.

2.11 F11 – Visualizar listado de ofertas.

2.11.1 Acceso

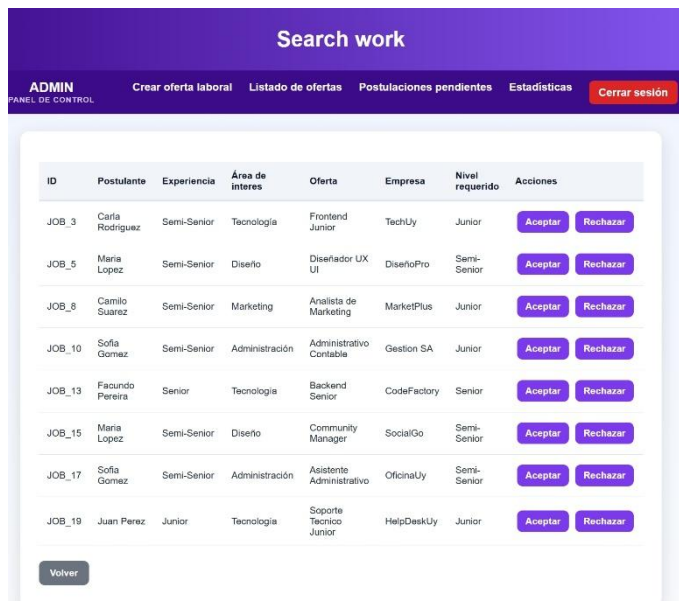
Puede acceder el administrador.

2.11.2 Descripción

El administrador podrá visualizar un listado completo de todas las ofertas laborales registradas en el sistema, incluyendo ofertas activas, inactivas y cerradas.

Desde esta pantalla podrá consultar la información básica de cada oferta y acceder a las opciones de editar o cerrar una oferta.

2.11.3 Interfaz de usuario



2.11.4 Validaciones

- Se deberán mostrar todas las ofertas del sistema.
- El estado de cada oferta deberá visualizarse correctamente.
- Las ofertas cerradas no deberán eliminarse del listado.
- Las opciones de editar y cerrar deberán aplicarse sobre la oferta seleccionada.

2.12 F12 – Procesar postulaciones.

2.12.1 Acceso

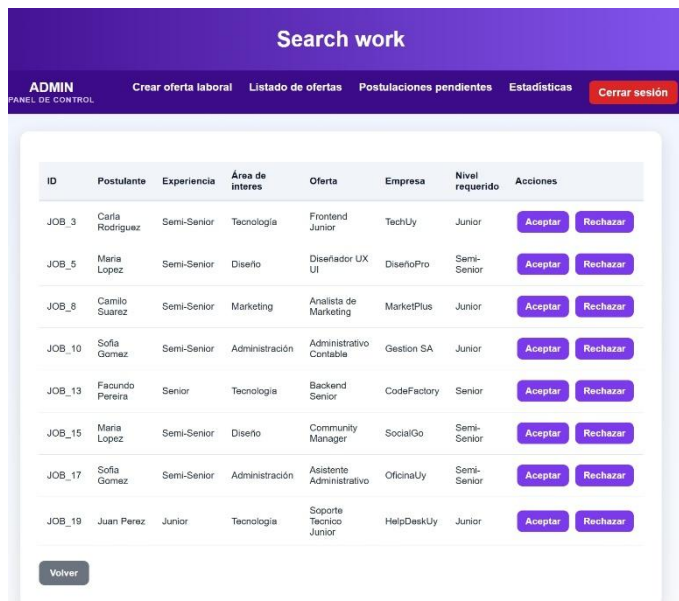
Puede acceder el administrador.

2.12.2 Descripción

El administrador podrá visualizar todas las postulaciones pendientes y procesarlas individualmente.

Al procesar una postulación, el administrador podrá aceptarla o rechazarla. El sistema realizará automáticamente las verificaciones correspondientes sobre el estado de la oferta y las vacantes disponibles.

2.12.3 Interfaz de usuario



2.12.4 Validaciones

- El sistema deberá actualizar correctamente el estado de la postulación.
- Si se cubren todas las vacantes de una oferta, la oferta deberá pasar automáticamente a estado “Inactiva”.
- Si se alcanza el límite de postulaciones, la oferta deberá pasar automáticamente a estado “Inactiva”.
- Si se cubren todas las vacantes, las demás postulaciones pendientes de esa oferta deberán rechazarse automáticamente.
- El sistema deberá informar:
 - si la postulación fue aceptada o rechazada,
 - si la oferta cambió de estado,
 - y cuántas postulaciones fueron rechazadas automáticamente. Principio del formulario

2.13 F13 – Visualizar estadísticas.

2.13.1 Acceso

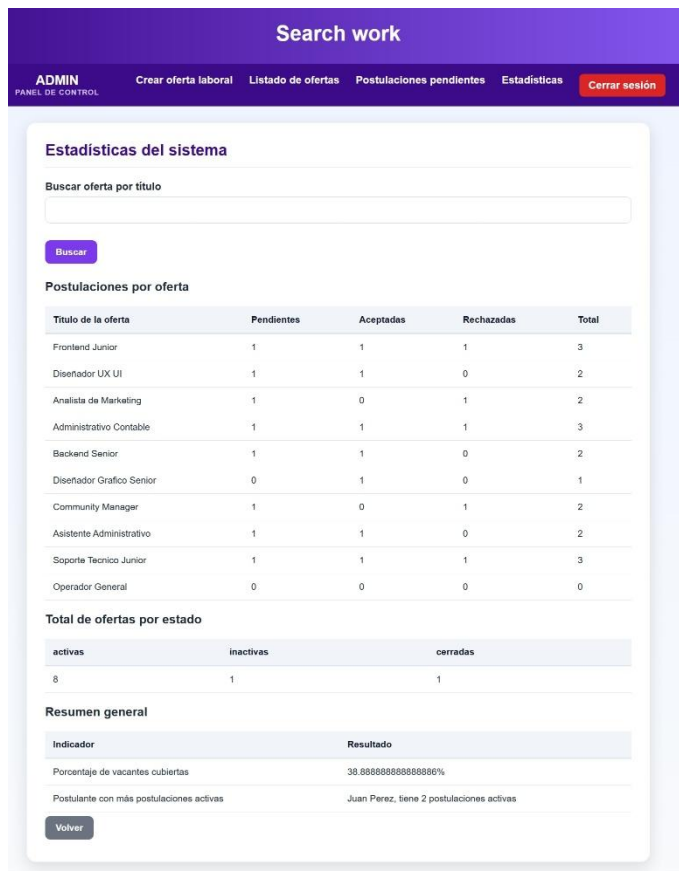
Puede acceder el administrador.

2.13.2 Descripción

El administrador podrá visualizar estadísticas generales del sistema relacionadas con las ofertas laborales y las postulaciones realizadas.

La información incluirá estadísticas sobre el estado de las ofertas, cantidad de postulaciones y vacantes cubiertas.

2.13.3 Interfaz de usuario



2.13.4 Validaciones

- La tabla deberá mostrar correctamente:
- cantidad de postulaciones pendientes,
- aceptadas,
- rechazadas,
- y total de postulaciones por oferta.
- El buscador deberá permitir buscar ofertas por título.

- Los totales de ofertas por estado deberán calcularse correctamente.
- El porcentaje de vacantes cubiertas deberá calcularse correctamente.
- El sistema deberá mostrar correctamente el postulante con más postulaciones activas.

3. Casos de prueba.

Funcionalidad	Caso de prueba	Datos de entrada	Salida esperada	Resultado obtenido
F01 – Registro	Usuario vacío	Usuario: ""	Mostrar error de usuario obligatorio	Cumple
F01 – Registro	Usuario menor a 5 caracteres	Usuario: "ana"	Mostrar error	Cumple
F01 – Registro	Usuario repetido	Usuario: "naza", existiendo "Naza"	Mostrar error	Cumple
F01 – Registro	Contraseña inválida	Menor a 5 caracteres, sin mayúscula, sin minúscula o sin número	Mostrar el error correspondiente	Cumple
F01 – Registro	Registro correcto	Datos válidos	Registrar correctamente al postulante	Cumple
F02 – Inicio de sesión	Credenciales incorrectas	Usuario inexistente o contraseña incorrecta	Mostrar credenciales incorrectas	Cumple

F02 – Inicio de sesión	Inicio correcto postulante	Usuario y contraseña válidos	Ingresar al menú de postulante	Cumple
F02 – Inicio de sesión	Inicio correcto administrador	Usuario y contraseña válidos	Ingresar al menú de administrador	Cumple
F03 – Cierre de sesión	Cierre correcto	Usuario con sesión iniciada	Cerrar sesión y volver al login	Cumple
F04 – Visualizar ofertas	Filtrado de ofertas	Oferta compatible, incompatible, cerrada y ya postulada	Mostrar solo ofertas disponibles	Cumple
F04 – Visualizar ofertas	Filtro por área	Área Tecnología	Mostrar por defecto ofertas del área	Cumple
F05 – Postularse	Postulación correcta	Oferta válida	Crear postulación pendiente	Cumple
F05 – Postularse	Postulación inválida	Oferta repetida, inactiva o incompatible	No permitir la postulación	Cumple
F06 – Mis postulaciones	Ver postulaciones	Usuario con y sin postulaciones	Mostrar sus postulaciones o informar que no tiene	Cumple
F07 – Ofertas destacadas	Visualización	Oferta destacada y no destacada	Mostrar solo las destacadas	Cumple

F07 – Ofertas destacadas	Postulación	Oferta destacada válida	Crear postulación pendiente	Cumple
F08 – Crear oferta	Datos inválidos	Campos vacíos, vacantes mayores al límite o valores inválidos	Mostrar mensaje de error	Cumple
F08 – Crear oferta	Creación correcta	Datos válidos	Crear oferta con estado Activa	Cumple
F09 – Editar oferta	Edición	Datos válidos e inválidos	Actualizar información o mostrar error	Cumple
F10 – Cerrar oferta	Cierre correcto	Oferta seleccionada	Cambiar estado a Cerrada	Cumple
F11 – Listado de ofertas	Visualización	Administrador logueado	Mostrar listado de ofertas con su estado	Cumple
F12 – Procesar postulaciones	Procesamiento	Aceptar, rechazar, cubrir vacantes y rechazo automático	Actualizar estados correctamente	Cumple
F13 – Estadísticas	Estadísticas generales	Datos del sistema	Mostrar tabla, búsqueda, totales, porcentaje y postulante con más	Cumple

			postulaciones activas	
--	--	--	--------------------------	--

1.3. Contexto de uso.

Herramienta utilizada: ChatGPT (OpenAI).

La herramienta fue utilizada como apoyo durante el desarrollo del obligatorio para:

- Redacción y mejora de la documentación.
- Organización y descripción de las funcionalidades.
- Corrección de estilo, ortografía y formato.
- Generación de ejemplos de validaciones.
- Elaboración de casos de prueba (testing).
- Explicación de conceptos y resolución de dudas puntuales sobre la implementación.

Toda la información y el contenido generado fueron revisados, adaptados, corregidos y validados manualmente por los integrantes del equipo antes de ser incorporados al trabajo final.

Prompts utilizados

1. "Ayúdame a redactar la documentación de las funcionalidades del obligatorio de Programación 1 con un lenguaje claro y formal."
2. "Revisá el texto y corregí la ortografía, gramática y redacción sin cambiar el contenido."
3. "Organizá las funcionalidades del sistema en un formato más claro para la documentación."
4. "Generá ejemplos de casos de prueba para las validaciones del registro, inicio de sesión y creación de ofertas."
5. "Ayúdame a elaborar una tabla de casos de prueba con datos de entrada, salida esperada y resultado obtenido."
6. "Revisá este método y decime si cumple con la funcionalidad pedida en la letra del obligatorio."
7. "Revisá la documentación final y proponé mejoras de formato y presentación."

8. "Explicame como generar métodos de búsqueda en arrays , métodos de Cálculos y como ubicarme entre objetos y arrays dentro de ellos .
No me des las soluciones , quiero tomar un caso diferente en base al que tenemos e ir aprendiendo como hacerlo ."
9. "Con que comando se indenta el código en visual code ?"
10. "Te voy a pasar los métodos generados hasta ahora y quiero que me hagas preguntas de ellos para saber si entiendo cada cosa, no avancemos de método hasta que no estes seguro de que puedo responder todas tus preguntas de dicho método"
11. "Bríndame un CSS para este HTML "

```
let adminAutoincrement = 1;
```

```
class Admin {
```

```
  #id;
```

```
  constructor(usuario, password, nombre){
```

```
    this.#id = adminAutoincrement;
```

```
    adminAutoincrement++;
```

```
    this.usuario = usuario;
```

```
    this.password = password;
```

```
    this.nombre = nombre;
```

```
  }
```

```
getId(){  
  
    return this.#id;  
  
}  
  
}
```

este metodo lo uso cuando el admin edita una oferta desde el listado de ofertas.

recibe los datos nuevos y actualiza los atributos del objeto sin crear una oferta nueva.

```
*/  
  
    editarOferta(titulo, empresa, descripcion, nivel, area, limitePostulaciones,  
cantidadVacantes, destacada) {  
  
    this.titulo = titulo;  
  
    this.empresa = empresa;  
  
    this.descripcion = descripcion;  
  
    this.nivel = nivel;  
  
    this.area = area;  
  
    this.limitePostulaciones = limitePostulaciones;  
  
    this.cantidadVacantes = cantidadVacantes;  
  
    this.destacada = destacada;  
  
}  
  
}
```

```
let postulacionAutoincrement = 1;

// como parámetro estoy dando mis 2 objetos

class Postulacion {

    #id

    constructor(postulante, ofertaLaboral){

        this.#id = `JOB_${postulacionAutoincrement}`;

        postulacionAutoincrement++;

        this.postulante = postulante;

        this.ofertaLaboral = ofertaLaboral;

        this.estado = "pendiente";

    }

    getId() {

        return this.#id;

    }

}
```

```
let postulanteAutoincrement = 1;

class Postulante {

  #id;

  constructor(usuario, password, nombre, experiencia, area) {

    this.#id = postulanteAutoincrement++;

    this.usuario = usuario;

    this.password = password;

    this.nombre = nombre;

    this.experiencia = experiencia;

    this.area = area;

  }

  getId() {

    return this.#id;

  }

}
```

```
class Sistema {  
  
    #tipoUser;  
  
    constructor() {  
  
        this.#tipoUser = null;  
  
        this.postulantes = [];  
  
        this.admins = [];  
  
        this.ofertas = [];  
  
        this.postulaciones = [];  
  
  
        this.usuarioLogueado = null;  
  
  
        this.precargaDatos();  
    }  
  
  
    //PRECARGA DE DATOS  
  
    precargaDatos() {
```

```
// ADMINISTRADORES
```

```
this.registrarAdmin("adminrodri", "Rodri123", "Rodri");
```

```
this.registrarAdmin("adminingerard", "Gerard123", "El Gerry");
```

```
this.registrarAdmin("adminnaza", "Naza123", "Naza");
```

```
// POSTULANTES
```

```
this.registrarPostulante("juanp", "Juan123", "Juan Perez", "Junior", "Tecnología");
```

```
    this.registrarPostulante("maria", "Maria123", "Maria Lopez", "Semi-Senior",  
"Diseño");
```

```
    this.registrarPostulante("pedrog", "Pedro123", "Pedro Garcia", "Senior",  
"Marketing");
```

```
    this.registrarPostulante("lucias", "Lucia123", "Lucia Silva", "Junior",  
"Administración");
```

```
    this.registrarPostulante("carlar", "Carla123", "Carla Rodriguez", "Semi-Senior",  
"Tecnología");
```

```
    this.registrarPostulante("matias", "Matias123", "Matias Fernandez", "Senior",  
"Diseño");
```

```
    this.registrarPostulante("flores", "Flor123", "Flores Diaz", "Junior",  
"Marketing");
```

```
this.registrarPostulante("sofiag", "Sofia123", "Sofia Gomez", "Semi-Senior",  
"Administración");
```

```
this.registrarPostulante("nicolas", "Nico123", "Nicolas Castro", "Senior",  
"Tecnología");
```

```
this.registrarPostulante("valentina", "Vale123", "Valentina Ruiz", "Junior",  
"Otros");
```

```
this.registrarPostulante("camilo", "Camilo123", "Camilo Suarez", "Semi-Senior",  
"Marketing");
```

```
this.registrarPostulante("agusm", "Agus123", "Agustina Molina", "Senior",  
"Administración");
```

```
this.registrarPostulante("martina", "Martin123", "Martin Acosta", "Junior",  
"Diseño");
```

```
this.registrarPostulante("biancam", "Bianca123", "Bianca Mendez", "Semi-Senior",  
"Otros");
```

```
this.registrarPostulante("facundo", "Facu123", "Facundo Pereira", "Senior",  
"Tecnología");
```

// OFERTAS LABORALES

```
this.crearOferta("Frontend Junior", "TechUy", "HTML CSS y JavaScript", "Junior",  
"Tecnología", 5, 2, true);
```

```
this.crearOferta("Diseñador UX UI", "DiseñoPro", "Diseño de interfaces web",  
"Semi-Senior", "Diseño", 4, 2, true);
```

```
this.crearOferta("Analista de Marketing", "MarketPlus", "Campañas digitales",  
"Junior", "Marketing", 3, 1, false);
```

```
this.crearOferta("Administrativo Contable", "Gestion SA", "Tareas administrativas",  
"Junior", "Administración", 4, 2, false);
```

```
this.crearOferta("Backend Senior", "CodeFactory", "Desarrollo backend", "Senior",  
"Tecnología", 5, 2, false);
```

```
this.crearOferta("Diseñador Grafico Senior", "Creativa", "Piezas graficas", "Senior",  
"Diseño", 3, 1, false);
```

```
this.crearOferta("Community Manager", "SocialGo", "Gestion de redes sociales",  
"Semi-Senior", "Marketing", 4, 2, true);
```

```
this.crearOferta("Asistente Administrativo", "OficinaUy", "Soporte administrativo",  
"Semi-Senior", "Administración", 3, 1, false);
```

```
this.crearOferta("Soporte Tecnico Junior", "HelpDeskUy", "Atencion a usuarios",  
"Junior", "Tecnología", 6, 3, false);
```

```
this.crearOferta("Operador General", "ServiciosYa", "Tareas generales", "Semi-  
Senior", "Otros", 4, 2, false);
```

```
// CAMBIO DE ESTADOS DE ALGUNAS OFERTAS
```

```
this.ofertas[5].inactivarOferta();
```

```
this.ofertas[9].cerrarOferta();
```

```
// POSTULACIONES
```

```
this.registrarPostulacion(this.postulantes[0], this.ofertas[0], "aceptada");
```

```
this.registrarPostulacion(this.postulantes[4], this.ofertas[0], "pendiente");
```

this.registrarPostulacion(this.postulantes[14], this.ofertas[0], "rechazada");

this.registrarPostulacion(this.postulantes[1], this.ofertas[1], "pendiente");

this.registrarPostulacion(this.postulantes[12], this.ofertas[1], "aceptada");

this.registrarPostulacion(this.postulantes[6], this.ofertas[2], "rechazada");

this.registrarPostulacion(this.postulantes[10], this.ofertas[2], "pendiente");

this.registrarPostulacion(this.postulantes[3], this.ofertas[3], "aceptada");

this.registrarPostulacion(this.postulantes[7], this.ofertas[3], "pendiente");

this.registrarPostulacion(this.postulantes[11], this.ofertas[3], "rechazada");

this.registrarPostulacion(this.postulantes[8], this.ofertas[4], "aceptada");

this.registrarPostulacion(this.postulantes[14], this.ofertas[4], "pendiente");

this.registrarPostulacion(this.postulantes[5], this.ofertas[5], "aceptada");

this.registrarPostulacion(this.postulantes[1], this.ofertas[6], "pendiente");

this.registrarPostulacion(this.postulantes[2], this.ofertas[6], "rechazada");

this.registrarPostulacion(this.postulantes[7], this.ofertas[7], "pendiente");

this.registrarPostulacion(this.postulantes[11], this.ofertas[7], "aceptada");

```
    this.registrarPostulacion(this.postulantes[0], this.ofertas[8], "pendiente");  
  
    this.registrarPostulacion(this.postulantes[4], this.ofertas[8], "rechazada");  
  
    this.registrarPostulacion(this.postulantes[8], this.ofertas[8], "aceptada");  
  
}
```

// Crea un administrador y lo agrega al sistema.

```
registrarAdmin(usuario, contrasenia, nombre) {  
  
    let nuevoAdmin = new Admin(usuario, contrasenia, nombre);  
  
    this.admins.push(nuevoAdmin);  
  
}
```

// Valida los datos ingresados y registra un nuevo postulante.

```
registrarPostulante(usuario, password, nombre, experiencia, area) {  
  
    let respuesta = validarRegistroPostulante(  
  
        usuario,  
  
        password,  
  
        nombre,  
  
        experiencia,
```

```

        area,

        this

    );

    if (respuesta === "") {

        let nuevoPostulante = new Postulante(

            usuario,

            password,

            nombre,

            experiencia,

            area

        );

        this.postulantes.push(nuevoPostulante);

        respuesta = "registro procesado";

    }

    return respuesta;

}

```

```
// Crea una postulación con el estado recibido por parámetro.
```

```
registrarPostulacion(postulante, oferta, estado) {  
  
    let nuevaPostulacion = new Postulacion(postulante, oferta);  
  
    nuevaPostulacion.estado = estado;  
  
    this.postulaciones.push(nuevaPostulacion);  
  
}
```

```
// Valida y crea una nueva oferta laboral.
```

```
    crearOferta(titulo, empresa, descripcion, nivel, area, limitePostulaciones,  
cantidadVacantes, destacada) {
```

```
let respuesta = validarOferta(titulo, empresa, descripcion, nivel, area,  
limitePostulaciones, cantidadVacantes, destacada);
```

```
if (respuesta === "") {
```

```
    let nuevaOferta = new OfertaLaboral(  
  
        titulo,  
  
        empresa,  
  
        descripcion,  
  
        nivel,  
  
        area,  
  
        limitePostulaciones,
```

```

        cantidadVacantes,

        destacada

    );

    this.ofertas.push(nuevaOferta);

    respuesta = "Oferta creada correctamente";

}

return respuesta;

}

// Verifica las credenciales e inicia sesión según el tipo de usuario.

login(usuario, password) {

    let posicionAdmin = findCasero(this.admins, "usuario", usuario);

    let posicionPostulante = findCasero(this.postulantes, "usuario", usuario);

    if (posicionAdmin !== -1) {

        let adminEncontrado = this.admins[posicionAdmin];

```

```
if (adminEncontrado.password === password) {  
  
    this.usuarioLogueado = adminEncontrado;  
  
    this.#tipoUser = "Admin";  
  
    return true;  
  
}  
  
return false;  
  
}  
  
if (posicionPostulante !== -1) {  
  
    let postulanteEncontrado = this.postulantes[posicionPostulante];  
  
    if (postulanteEncontrado.password === password) {  
  
        this.usuarioLogueado = postulanteEncontrado;  
  
        this.#tipoUser = "Postulante";  
  
        return true;  
  
    }  
  
    return false;  
  
}
```

```

        return false;
    }

    // Devuelve el tipo del usuario que tiene la sesión iniciada.
    getTipoUser() {
        return this.#tipoUser;
    }

    // Cierra la sesión actual.
    cerrarSesion() {
        this.usuarioLogueado = null;
        this.#tipoUser = null;
    }

    // Busca una oferta por su id y la devuelve.
    findOfertaById(idBuscado) {
        for (let i = 0; i < this.ofertas.length; i++) {

            if (this.ofertas[i].getId() === idBuscado) {

                return this.ofertas[i];
            }
        }
    }

```

```
    }  
}
```

```
    return null;  
}
```

// Verifica si un postulante ya se postuló a una oferta.

```
yaSePostulo(postulante, oferta) {  
    for (let i = 0; i < this.postulaciones.length; i++) {  
        let postulacionActual = this.postulaciones[i];  
  
        if (  
            postulacionActual.postulante === postulante &&  
            postulacionActual.ofertaLaboral === oferta  
        ) {  
            return true;  
        }  
    }  
  
    return false;  
}
```

```
// Cuenta cuántas postulaciones tiene una oferta.
```

```
contarPostulacionesOferta(oferta) {
```

```
    let contador = 0;
```

```
    for (let i = 0; i < this.postulaciones.length; i++) {
```

```
        if (this.postulaciones[i].ofertaLaboral === oferta) {
```

```
            contador++;
```

```
        }
```

```
    }
```

```
    return contador;
```

```
}
```

```
// Verifica si el nivel de experiencia es compatible con la oferta.
```

```
expCompatible(postulante, oferta) {
```

```
    if (
```

```
        postulante.experiencia === "Senior" &&
```

```
        (oferta.nivel === "Senior" || oferta.nivel === "Semi-Senior")
```

```
    ) {
```

```

        return true;
    }

    if (postulante.experiencia === "Semi-Senior") {

        return true;
    }

    if (

        postulante.experiencia === "Junior" &&

        (oferta.nivel === "Junior" || oferta.nivel === "Semi-Senior")

    ) {

        return true;
    }

    return false
}

// Realiza una nueva postulación si cumple las condiciones necesarias.

postularse(postulante, oferta) {

    if (validarOfertaParaPostulante(postulante, oferta)) {

        this.registrarPostulacion(postulante, oferta, "pendiente");
    }
}

```

```

        if(this.contarPostulacionesOferta(oferta)>= oferta.limitePostulaciones){

            oferta.inactivarOferta();

        }

        return "Postulación realizada correctamente";

    }

    return "No es posible postularse a esta oferta";

}

// Obtiene todas las postulaciones realizadas por un postulante.

obtenerMisPostulaciones(postulante) {

    let resultado = [];

    for (let i = 0; i < this.postulaciones.length; i++) {

        let postulacionActual = this.postulaciones[i];

        if (postulacionActual.postulante === postulante) {

            resultado.push(postulacionActual);

        }

    }

```

```
    }  
  
    return resultado;  
  
}
```

// Devuelve únicamente las postulaciones pendientes.

```
obtenerPostulacionesPendientes() {  
  
    let resultado = [];  
  
    for (let i = 0; i < this.postulaciones.length; i++) {  
  
        let postulacionActual = this.postulaciones[i];  
  
        if (postulacionActual.estado === "pendiente") {  
  
            resultado.push(postulacionActual);  
  
        }  
  
    }  
  
    return resultado;  
  
}
```

// Procesa una postulación aceptándola o rechazándola.

```
procesarPostulacion(idPostulacion, accion) {  
  
    let postulacionProcesada = null;  
  
    for (let i = 0; i < this.postulaciones.length; i++) {  
  
        let postulacionActual = this.postulaciones[i];  
  
        if (postulacionActual.getId() === idPostulacion) {  
  
            postulacionActual.estado = accion;  
  
            postulacionProcesada = postulacionActual;  
  
        }  
    }  
  
    if (postulacionProcesada === null) {  
  
        return "No se pudo procesar la postulación";  
    }  
  
    if (accion === "rechazada") {  
  
        return "Postulación rechazada correctamente";  
    }  
}
```

```

let oferta = postulacionProcesada.ofertaLaboral;

let rechazadasAutomaticamente = 0;

let cambioEstadoOferta = false;

let aceptadas = this.contarPostulacionesPorEstado(oferta, "aceptada");

let totalPostulaciones = this.contarPostulacionesOferta(oferta);

    if (aceptadas >= oferta.cantidadVacantes || totalPostulaciones >=
oferta.limitePostulaciones) {

        oferta.inactivarOferta();

        cambioEstadoOferta = true;

for (let i = 0; i < this.postulaciones.length; i++) {

    let postulacionActual = this.postulaciones[i];

    if (

        postulacionActual.ofertaLaboral === oferta &&

        postulacionActual.estado === "pendiente"

    ) {

        postulacionActual.estado = "rechazada";

```

```

        rechazadasAutomaticamente++;

    }

}

}

let mensaje = "Postulación aceptada correctamente";

if (cambioEstadoOferta === true) {

    if (aceptadas >= oferta.cantidadVacantes && totalPostulaciones >=
oferta.limitePostulaciones) {

        mensaje += "<br>La oferta pasó a estado Inactiva porque se cubrieron todas las
vacantes y se alcanzó el límite de postulaciones.";

    } else if (aceptadas >= oferta.cantidadVacantes) {

        mensaje += "<br>La oferta pasó a estado Inactiva porque se cubrieron todas las
vacantes.";

    } else {

        mensaje += "<br>La oferta pasó a estado Inactiva porque se alcanzó el límite de
postulaciones.";

    }

}

mensaje += "<br>Postulaciones rechazadas automáticamente: " +
rechazadasAutomaticamente;

```

```

    }

    return mensaje;
}

// Obtiene las estadísticas de postulaciones por oferta.
obtenerPostulacionesPorOferta(textoBuscado){

    let resultado = [];

    for (let i = 0; i < this.ofertas.length; i++) {

        let ofertaActual = this.ofertas[i];

        if (!ofertaActual.titulo.toLowerCase().includes(textoBuscado)) {

            continue;

        }

        let pendientes = 0;

        let aceptadas = 0;

        let rechazadas = 0;

        for (let j = 0; j < this.postulaciones.length; j++) {

```

```

let postulacionActual = this.postulaciones[j];

if (ofertaActual === postulacionActual.ofertaLaboral) {

    if (postulacionActual.estado === "pendiente") {

        pendientes++;

    } else if (postulacionActual.estado === "aceptada") {

        aceptadas++;

    } else if (postulacionActual.estado === "rechazada") {

        rechazadas++;

    }

}

}

}

resultado.push({

    titulo: ofertaActual.titulo,

    pendientes: pendientes,

    aceptadas: aceptadas,

    rechazadas: rechazadas,

    total: pendientes + aceptadas + rechazadas

});

}

```

```

    return resultado;

}

// Cuenta las ofertas según su estado.

obtenerTotalesOfertasPorEstado() {

    let activas = 0;

    let inactivas = 0;

    let cerradas = 0;

    this.ofertas.forEach(function(ofertaActual) {

        if (ofertaActual.getEstado() === "Activa") {

            activas++;

        } else if (ofertaActual.getEstado() === "Inactiva") {

            inactivas++;

        } else if (ofertaActual.getEstado() === "Cerrada") {

            cerradas++;

        }

    })

```

```

    });

    return {

        activas: activas,

        inactivas: inactivas,

        cerradas: cerradas

    };

}

// Calcula el porcentaje de vacantes cubiertas.

obtenerPorcentajeVacantesCubiertas() {

    let vacantesCubiertas = 0;

    let cantidadVacantes = 0;

    this.ofertas.forEach(function(ofertaActual) {

        cantidadVacantes += Number(ofertaActual.cantidadVacantes);

    });

    this.postulaciones.forEach(function(postulacionActual) {

        if (postulacionActual.estado === "aceptada") {

```

```

        vacantesCubiertas++;

    }

});

return vacantesCubiertas * 100 / cantidadVacantes;

}

// Busca el postulante con más postulaciones en ofertas activas.
obtenerPostulanteMasPostulacionesActivas() {

    let postulanteMayor = null;

    let mayorCantidad = 0;

    for (let i = 0; i < this.postulantes.length; i++) {

        let postulanteActual = this.postulantes[i];

        let cantidad = 0;

        for (let j = 0; j < this.postulaciones.length; j++) {

            let postulacionActual = this.postulaciones[j];

```

```

    if (
        postulacionActual.postulante === postulanteActual &&
        postulacionActual.ofertaLaboral.getEstado() === "Activa"
    ) {
        cantidad++;
    }
}

if (cantidad > mayorCantidad) {
    mayorCantidad = cantidad;
    postulanteMayor = postulanteActual;
}

}

if (postulanteMayor === null) {
    return {
        postulante: { nombre: "No hay postulaciones activas" },
        cantidad: 0
    };
}

```

```
return {  
  
    postulante: postulanteMayor,  
  
    cantidad: mayorCantidad  
  
};  
  
}
```

// Devuelve todas las ofertas registradas.

```
obtenerTodasLasOfertas() {  
  
    return this.ofertas;  
  
}
```

// Cierra una oferta a partir de su id.

```
cerrarOfertaPorId(idOferta) {  
  
    let oferta = this.findOfertaById(idOferta);  
  
    if (oferta !== null) {  
  
        oferta.cerrarOferta();  
  
    }  
  
}
```

```
}
```

```
// Modifica los datos de una oferta existente.
```

```
editarOfertaPorId(idOferta, titulo, empresa, descripcion, nivel, area, limitePostulaciones,  
cantidadVacantes, destacada) {
```

```
    let respuesta = validarOferta(titulo, empresa, descripcion, nivel, area,  
limitePostulaciones, cantidadVacantes, destacada);
```

```
    if (respuesta === "") {
```

```
        let oferta = this.findOfertaById(idOferta);
```

```
        if (oferta !== null) {
```

```
            oferta.editarOferta(titulo, empresa, descripcion, nivel, area, limitePostulaciones,  
cantidadVacantes, destacada);
```

```
            respuesta = "Oferta editada correctamente";
```

```
        } else {
```

```
            respuesta = "No se pudo encontrar la oferta";
```

```
        }
```

```
    }
```

```

    return respuesta;
}

// Obtiene las ofertas aplicando filtros de área y destacadas.

obtenerOfertasFiltradas(postulante, filtroArea, soloDestacadas) {

    let resultado = [];

    for (let i = 0; i < this.ofertas.length; i++) {

        let ofertaActual = this.ofertas[i];

        let cumpleArea = filtroArea !== "area" || ofertaActual.area === postulante.area;

        let cumpleDestacada = soloDestacadas === false || ofertaActual.destacada === true;

        if (validarOfertaParaPostulante(postulante, ofertaActual) && cumpleArea &&
cumpleDestacada){

            resultado.push(ofertaActual);

        }

    }

    return resultado;
}

```

// Obtiene las ofertas disponibles para un postulante.

```
obtenerOfertasParaPostulante(postulante, filtro) {  
  
    return this.obtenerOfertasFiltradas(postulante, filtro, false);  
  
}
```

// Obtiene las ofertas para mostrar en la tabla principal.

```
obtenerOfertasParaTablaPostulante(postulante) {  
  
    return this.obtenerOfertasFiltradas(postulante, "todas", false);  
  
}
```

// Obtiene únicamente las ofertas destacadas.

```
obtenerOfertasParaTablaDestacadas(postulante) {  
  
    return this.obtenerOfertasFiltradas(postulante, "todas", true);  
  
}
```

// Cuenta postulaciones de una oferta según un estado específico.

```
contarPostulacionesPorEstado(oferta, estado) {  
  
    let contador = 0;  
  
    for (let i = 0; i < this.postulaciones.length; i++) {
```

```

    let postulacionActual = this.postulaciones[i];

    if (
        postulacionActual.ofertaLaboral === oferta &&
        postulacionActual.estado === estado
    ) {
        contador++;
    }
}

return contador;
}

// Busca una oferta por id y realiza la postulación.
postularsePorId(postulante, idOferta) {
    let ofertaSeleccionada = this.findOfertaById(idOferta);

    if (ofertaSeleccionada !== null) {
        return this.postularse(postulante, ofertaSeleccionada);
    }
}

```

```

    return "No se encontró la oferta";

}

}

// Inicializa el menú principal del administrador y conecta sus botones.

function initAdmin() {

    let btnCrearOferta = document.querySelector("#btnCrearOferta");

    let btnCerrarSesionAdmin = document.querySelector("#btnCerrarSesionAdmin");

    let btnListadoOfertasAdmin =
document.querySelector("#btnVerListadoOfertasAdmin");

    let btnPostulacionesPendientesAdmin =
document.querySelector("#btnVerPostulacionesPendientes");

    let btnEstadísticas = document.querySelector("#btnVerEstadisticasAdmin");

    btnListadoOfertasAdmin.addEventListener("click", listadoOfertasAdmin);

    btnPostulacionesPendientesAdmin.addEventListener("click",
postulacionesPendientesAdmin);

    btnEstadísticas.addEventListener("click", estadisticas);

    btnCrearOferta.addEventListener("click", crearOferta);

    btnCerrarSesionAdmin.addEventListener("click", cerrarSesionAdmin);

}

```

```
// Vuelve al listado de ofertas del administrador.
```

```
function volverAdmin() {  
  
    irA("view-table-ofertas-admin", initListadoOfertasAdmin);  
  
}
```

```
// Cierra la sesión actual y vuelve al login.
```

```
function cerrarSesionAdmin() {  
  
    sistema.cerrarSesion();  
  
    irA("view-login", initLogin);  
  
}
```

```
// Navega a la pantalla de creación de ofertas.
```

```
function crearOferta() {  
  
    irA("view-crear-oferta", initCrearOferta);  
  
}
```

```
// Navega al listado de ofertas.
```

```
function listadoOfertasAdmin() {  
  
    irA("view-table-ofertas-admin", initListadoOfertasAdmin);  
  
}
```

```
// Navega a la pantalla de postulaciones pendientes.
```

```
function postulacionesPendientesAdmin() {
```

```
    irA("view-postulaciones-pendientes", initPostulacionesPendientesAdmin);
```

```
}
```

```
// Navega a la pantalla de estadísticas.
```

```
function estadisticas() {
```

```
    irA("view-estadisticas-admin", initEstadistica);
```

```
}
```

```
// Inicializa la pantalla de creación de ofertas.
```

```
function initCrearOferta() {
```

```
    initAdmin();
```

```
    let btnGuardarOferta = document.querySelector("#btnGuardarOferta");
```

```
    let btnVolverMenuAdmin = document.querySelector("#btnVolverMenuAdmin1");
```

```
    btnGuardarOferta.addEventListener("click", guardarOferta);
```

```

    btnVolverMenuAdmin.addEventListener("click", volverAdmin);

}

// Obtiene los datos ingresados y crea una nueva oferta.

function guardarOferta() {

    let titulo = document.querySelector("#txtTituloOferta").value.trim();

    let empresa = document.querySelector("#txtEmpresaOferta").value.trim();

    let descripcion = document.querySelector("#txtDescripcionOferta").value.trim();

    let nivel = document.querySelector("#slcNivelOferta").value;

    let area = document.querySelector("#slcAreaOferta").value;

    let limitePostulaciones =
Number(document.querySelector("#txtLimitePostulaciones").value);

    let cantidadVacantes =
Number(document.querySelector("#txtCantidadVacantes").value);

    let destacada = document.querySelector("#slcOfertaDestacada").value;

    let esDestacada = false;

    if (destacada === "si") {

        esDestacada = true;

    }
}

```

```

let respuesta = sistema.crearOferta(

    titulo,

    empresa,

    descripcion,

    nivel,

    area,

    limitePostulaciones,

    cantidadVacantes,

    esDestacada

);

if (respuesta === "Oferta creada correctamente") {

    irA("view-table-ofertas-admin", initListadoOfertasAdmin);

} else {

    document.querySelector("#pCrearOferta").innerHTML = respuesta;

}

}

// Inicializa la pantalla de estadísticas y carga la información.

function initEstadistica() {

```

```

initAdmin();

let txtBuscarOfertaEstadistica =
document.querySelector("#txtBuscarOfertaEstadistica");

let btnBuscarOfertaEstadistica =
document.querySelector("#btnBuscarOfertaEstadistica");

let btnVolverMenuAdmin4 = document.querySelector("#btnVolverMenuAdmin4");

btnBuscarOfertaEstadistica.addEventListener("click", buscarOfertaEstadistica);

btnVolverMenuAdmin4.addEventListener("click", volverAdmin);

mostrarOfertaEstadistica();

mostrarTotalesEstadistica();

mostrarResumenGeneral();

}

// Muestra las estadísticas de postulaciones por oferta.

function mostrarOfertaEstadistica(textoBuscado = "") {

let tabla = document.querySelector("#tbodyPostulacionesPorOferta");

tabla.innerHTML = "";

```

```

let datos = sistema.obtenerPostulacionesPorOferta(textoBuscado);

if (datos.length === 0) {

    tabla.innerHTML = `<tr><td colspan="5">No hay ofertas registradas</td></tr>`;

}

for (let i = 0; i < datos.length; i++) {

    let fila = datos[i];

    tabla.innerHTML += `

        <tr>

            <td>${fila.titulo}</td>

            <td>${fila.pendientes}</td>

            <td>${fila.aceptadas}</td>

            <td>${fila.rechazadas}</td>

            <td>${fila.total}</td>

        </tr>

    `;

}

}

```

```
// Busca ofertas por título para las estadísticas.
```

```
function buscarOfertaEstadistica() {
```

```
    let txtBuscarOfertaEstadistica =  
document.querySelector("#txtBuscarOfertaEstadistica");
```

```
    let textoBuscado = txtBuscarOfertaEstadistica.value.toLowerCase().trim();
```

```
    mostrarOfertaEstadistica(textoBuscado);
```

```
}
```

```
// Muestra la cantidad de ofertas por estado.
```

```
function mostrarTotalesEstadistica() {
```

```
    let tbodyTotalOfertasPorEstado =  
document.querySelector("#tbodyTotalOfertasPorEstado");
```

```
    tbodyTotalOfertasPorEstado.innerHTML = "";
```

```
    let totales = sistema.obtenerTotalesOfertasPorEstado();
```

```
tbodyTotalOfertasPorEstado.innerHTML = `

<tr>

    <td>${totales.activas}</td>

    <td>${totales.inactivas}</td>

    <td>${totales.cerradas}</td>

</tr>`;
}

// Muestra el resumen general de estadísticas del sistema.

function mostrarResumenGeneral() {

    let tbodyResumenGeneral = document.querySelector("#tbodyResumenGeneral");

    let porcentaje = sistema.obtenerPorcentajeVacantesCubiertas();

    let resultado = sistema.obtenerPostulanteMasPostulacionesActivas();

    tbodyResumenGeneral.innerHTML = `

    <tr>

        <td>Porcentaje de vacantes cubiertas</td>

        <td>${porcentaje}%</td>

    </tr>

    <tr>
```

```

        <td>Postulante con más postulaciones activas</td>

        <td>${resultado.postulante.nombre}, tiene ${resultado.cantidad} postulaciones
activas</td>

    </tr>

    `;

}

```

```

// Inicializa la pantalla de listado de ofertas.

```

```

function initListadoOfertasAdmin() {

```

```

    initAdmin();

```

```

    let btnVolverMenuAdmin2 = document.querySelector("#btnVolverMenuAdmin2");

```

```

    mostrarListadoOfertasAdmin();

```

```

    btnVolverMenuAdmin2.addEventListener("click", volverAdmin);

```

```

}

```

```

// Muestra todas las ofertas registradas en el sistema.

```

```

function mostrarListadoOfertasAdmin() {

```

```
let tabla = document.querySelector("#tbodyListadoOfertasAdmin");
```

```
tabla.innerHTML = "";
```

```
let ofertas = sistema.obtenerTodasLasOfertas();
```

```
if (ofertas.length === 0) {
```

```
    tabla.innerHTML = `<tr><td colspan="8">No hay ofertas registradas</td></tr>`;
```

```
}
```

```
for (let i = 0; i < ofertas.length; i++) {
```

```
    //aca aplico el operador ternario
```

```
    let ofertaActual = ofertas[i];
```

```
    if (ofertaActual.getEstado() === "Activa"){
```

```
        tabla.innerHTML += `
```

```
            <tr>
```

```
                <td>${ofertaActual.getId()}</td>
```

```
                <td>${ofertaActual.titulo}</td>
```

```

<td>${ofertaActual.empresa}</td>

<td>${ofertaActual.nivel}</td>

<td>${ofertaActual.area}</td>

<td>${ofertaActual.destacada ? "si" : "no"}</td>

<td>${ofertaActual.getEstado()}</td>

<td>

<button    class="btnEditarOferta"    data-
id="${ofertaActual.getId()}">Editar</button>

<button    class="btnCerrarOferta"    data-
id="${ofertaActual.getId()}">Cerrar</button>

</td>

</tr>

`;

}

}

```

```
let botonesEditar = document.querySelectorAll(".btnEditarOferta");
```

```
let botonesCerrar = document.querySelectorAll(".btnCerrarOferta");
```

```
for (let i = 0; i < botonesEditar.length; i++) {
```

```
    botonesEditar[i].addEventListener("click", procesarEditarOferta);
```

```

    }

    for (let i = 0; i < botonesCerrar.length; i++) {

        botonesCerrar[i].addEventListener("click", procesarCierreOferta);

    }

}

// Cierra una oferta seleccionada desde el listado.

function procesarCierreOferta() {

    let idOferta = this.getAttribute("data-id");

    sistema.cerrarOfertaPorId(idOferta);

    mostrarListadoOfertasAdmin();

}

let idOfertaEditando = "";

```

```
// Guarda el id de la oferta seleccionada y abre la edición.
```

```
function procesarEditarOferta() {
```

```
    idOfertaEditando = this.getAttribute("data-id");
```

```
    irA("view-editar-oferta", initEditarOferta);
```

```
}
```

```
// Inicializa la pantalla de edición de ofertas y carga sus datos.
```

```
function initEditarOferta() {
```

```
    initAdmin();
```

```
    let btnGuardarCambiosOferta =  
document.querySelector("#btnGuardarCambiosOferta");
```

```
    let btnCancelarEditarOferta = document.querySelector("#btnCancelarEditarOferta");
```

```
    btnGuardarCambiosOferta.addEventListener("click", guardarCambiosOferta);
```

```
    btnCancelarEditarOferta.addEventListener("click", cancelarEditarOferta);
```

```

let oferta = sistema.findOfertaById(idOfertaEditando);

if (oferta !== null) {

    document.querySelector("#txtEditarTituloOferta").value = oferta.titulo;

    document.querySelector("#txtEditarEmpresaOferta").value = oferta.empresa;

    document.querySelector("#txtEditarDescripcionOferta").value = oferta.descripcion;

    document.querySelector("#slcEditarNivelOferta").value = oferta.nivel;

    document.querySelector("#slcEditarAreaOferta").value = oferta.area;

        document.querySelector("#txtEditarLimitePostulaciones").value    =
oferta.limitePostulaciones;

        document.querySelector("#txtEditarCantidadVacantes").value    =
oferta.cantidadVacantes;

    if (oferta.destacada === true) {

        document.querySelector("#slcEditarOfertaDestacada").value = "si";

    } else {

        document.querySelector("#slcEditarOfertaDestacada").value = "no";

    }

}

}

```

```
// Guarda los cambios realizados sobre una oferta.
```

```
function guardarCambiosOferta() {
```

```
    let titulo = document.querySelector("#txtEditarTituloOferta").value.trim();
```

```
    let empresa = document.querySelector("#txtEditarEmpresaOferta").value.trim();
```

```
        let                descripcion                =  
document.querySelector("#txtEditarDescripcionOferta").value.trim();
```

```
    let nivel = document.querySelector("#slcEditarNivelOferta").value;
```

```
    let area = document.querySelector("#slcEditarAreaOferta").value;
```

```
        let                limitePostulaciones                =  
Number(document.querySelector("#txtEditarLimitePostulaciones").value);
```

```
        let                cantidadVacantes                =  
Number(document.querySelector("#txtEditarCantidadVacantes").value);
```

```
    let destacada = document.querySelector("#slcEditarOfertaDestacada").value;
```

```
    let esDestacada = false;
```

```
    if (destacada === "si") {
```

```
        esDestacada = true;
```

```
    }
```

```

let respuesta = sistema.editarOfertaPorId(

    idOfertaEditando,

    titulo,

    empresa,

    descripcion,

    nivel,

    area,

    limitePostulaciones,

    cantidadVacantes,

    esDestacada

);

if (respuesta === "Oferta editada correctamente") {

    idOfertaEditando = "";

    irA("view-table-ofertas-admin", initListadoOfertasAdmin);

} else {

    document.querySelector("#pEditarOferta").innerHTML = respuesta;

}

}

// Cancela la edición y vuelve al listado de ofertas.

```

```

function cancelarEditarOferta() {

    idOfertaEditando = "";

    irA("view-table-ofertas-admin", initListadoOfertasAdmin);

}

// Inicializa la pantalla de postulaciones pendientes.

function initPostulacionesPendientesAdmin() {

    initAdmin();

    let btnVolverMenuAdmin3 = document.querySelector("#btnVolverMenuAdmin3");

    mostrarPostulacionesPendientes();

    btnVolverMenuAdmin3.addEventListener("click", volverAdmin);

}

// Muestra todas las postulaciones pendientes.

function mostrarPostulacionesPendientes() {

    let tabla = document.querySelector("#tbodyPostulacionesPendientes");

    tabla.innerHTML = "";

```

```

let postulacionesPendientes = sistema.obtenerPostulacionesPendientes();

for (let i = 0; i < postulacionesPendientes.length; i++) {

    let postulacionActual = postulacionesPendientes[i];

    tabla.innerHTML += `

        <tr>

            <td>${postulacionActual.getId()}</td>

            <td>${postulacionActual.postulante.nombre}</td>

            <td>${postulacionActual.postulante.experiencia}</td>

            <td>${postulacionActual.postulante.area}</td>

            <td>${postulacionActual.ofertaLaboral.titulo}</td>

            <td>${postulacionActual.ofertaLaboral.empresa}</td>

            <td>${postulacionActual.ofertaLaboral.nivel}</td>

            <td>

                <button class="btnAceptarPostulacion" data-
id="${postulacionActual.getId()}">Aceptar</button>

                <button class="btnRechazarPostulacion" data-
id="${postulacionActual.getId()}">Rechazar</button>

            </td>

        </tr>

```

```
`;  
}
```

```
let botonesAceptar = document.querySelectorAll(".btnAceptarPostulacion");
```

```
//LOS RECORREMOS Y CUANDO CONCIDEN CON LOS QUE HICIMOS CLICK  
LO RECONOCEMOS .( O NO SE COM LLAMARLO PERO COMO QUE NOS  
DAMOS CUENTA QUE ES EL , CAPAZ SE EEJCUTA O ESCUCHA )
```

```
for (let i = 0; i < botonesAceptar.length; i++) {  
  
    botonesAceptar[i].addEventListener("click", aceptarPostulacion);  
  
}
```

```
let botonesRechazar = document.querySelectorAll(".btnRechazarPostulacion");
```

```
for (let i = 0; i < botonesRechazar.length; i++) {  
  
    botonesRechazar[i].addEventListener("click", rechazarPostulacion);  
  
}  
  
}
```

```
// Procesa la aceptación de una postulación.
```

```
function aceptarPostulacion() {  
  
    let idPostulacion = this.getAttribute("data-id");
```

```
let mensaje = sistema.procesarPostulacion(idPostulacion, "aceptada");

document.querySelector("#pPostulacionesPendientes").innerHTML = mensaje;

mostrarPostulacionesPendientes();

}

// Procesa el rechazo de una postulación.

function rechazarPostulacion() {

    let idPostulacion = this.getAttribute("data-id");

    let mensaje = sistema.procesarPostulacion(idPostulacion, "rechazada");

    document.querySelector("#pPostulacionesPendientes").innerHTML = mensaje;

    mostrarPostulacionesPendientes();

}
```

```
//Loguin
```

```
function irRegistro() {  
  
    irA("view-registro", initRegistro);  
  
}
```

```
// Inicializa el menú del postulante y conecta los botones de navegación.
```

```
function initPostulante(){  
  
    let btnVerOfertas = document.querySelector("#btnVerOfertas");  
  
    let btnVerMisPostulaciones = document.querySelector("#btnVerMisPostulaciones");  
  
        let                btnVerOfertasDestacadas                =  
document.querySelector("#btnVerOfertasDestacadas");  
  
        let                btnCerrarSesionPostulante                =  
document.querySelector("#btnCerrarSesionPostulante");
```

```
btnVerOfertas.addEventListener("click", verOfertas);

btnVerMisPostulaciones.addEventListener("click", verMisPostulaciones);

btnVerOfertasDestacadas.addEventListener("click", verOfertasDestacadas);

btnCerrarSesionPostulante.addEventListener("click", cerrarSesionPostulante);

}
```

// Cierra la sesión del postulante y vuelve al login.

```
function cerrarSesionPostulante() {

    sistema.cerrarSesion();

    irA("view-login", initLogin);

}
```

// Navega a la pantalla de ofertas laborales disponibles.

```
function verOfertas() {

    irA("view-ofertas-postulante", initOfertasPostulante);

}
```

// Navega a la pantalla donde el postulante ve sus postulaciones.

```
function verMisPostulaciones() {

    irA("view-mis-postulaciones", initMisPostulaciones);

}
```

```
// Navega a la pantalla de ofertas destacadas.
```

```
function verOfertasDestacadas() {  
  
    irA("view-ofertas-destacadas", initOfertasDestacadas);  
  
}
```

```
// Vuelve a la pantalla principal de ofertas del postulante.
```

```
function volverMenuPostulante() {  
  
    irA("view-ofertas-postulante", initOfertasPostulante);  
  
}
```

```
// Inicializa la pantalla de mis postulaciones y muestra los datos.
```

```
function initMisPostulaciones() {  
  
    initPostulante();  
  
    let btnVolverMenuPostulante2 =  
document.querySelector("#btnVolverMenuPostulante2");  
  
    mostrarMisPostulaciones();  
  
    btnVolverMenuPostulante2.addEventListener("click", volverMenuPostulante);  
  
}
```

```
// Muestra en la tabla las postulaciones del usuario logueado.

function mostrarMisPostulaciones() {

    let tabla = document.querySelector("#tbodyListadoMisPos");

    tabla.innerHTML = "";

    //llamo al metodo desde aca y loguardo en una variable ( para recorrer unicamente las
    postu de los logueados y no todas las postu)

    let misPostulaciones = sistema.obtenerMisPostulaciones(sistema.usuarioLogueado);

    for(let i = 0; i < misPostulaciones.length; i++){

        let postulacionActual = misPostulaciones[i];

        tabla.innerHTML += `

            <tr>

                <td>${postulacionActual.getId()}</td>

                <td>${postulacionActual.ofertaLaboral.titulo}</td>

                <td>${postulacionActual.ofertaLaboral.empresa}</td>

                <td>${postulacionActual.ofertaLaboral.nivel}</td>

                <td>${postulacionActual.ofertaLaboral.area}</td>

                <td>${postulacionActual.ofertaLaboral.destacada ? "★" : "-"}</td>

        `
    }
}
```

```

        <td>${postulacionActual.estado}</td>

    </tr>

    `;

    }

}

// Inicializa la pantalla de ofertas destacadas y conecta sus eventos.

function initOfertasDestacadas() {

    initPostulante();

    let btnVolverMenuPostulante3 =
document.querySelector("#btnVolverMenuPostulante3");

    mostrarOfertasDestacadas();

    btnVolverMenuPostulante3.addEventListener("click", volverMenuPostulante);

}

// Muestra las ofertas destacadas disponibles para el postulante.

function mostrarOfertasDestacadas() {

```

```

let tabla = document.querySelector("#tbodyListadoDestacadas");

tabla.innerHTML = "";

let ofertasDestacadas =
sistema.obtenerOfertasParaTablaDestacadas(sistema.usuarioLogueado);

for (let i = 0; i < ofertasDestacadas.length; i++) {

    let ofertaActual = ofertasDestacadas[i];

    tabla.innerHTML += `

        <tr>

            <td>${ofertaActual.titulo}</td>

            <td>${ofertaActual.empresa}</td>

            <td>${ofertaActual.nivel}</td>

            <td>${ofertaActual.area}</td>

            <td>★</td>

            <td>${ofertaActual.descripcion}</td>

            <td>${ofertaActual.getEstado()}</td>

```

```

        <td>

        <button class="btnPostularme" data-id="{ofertaActual.getId()}">

            Postularme

        </button>

    </td>

</tr>

`;

}

let btnsPostularme = document.querySelectorAll(".btnPostularme");

for(let i = 0; i < btnsPostularme.length; i++){

    btnsPostularme[i].addEventListener("click", hacerPostulacionDestacada);

}

}

// Procesa la postulación desde la pantalla de ofertas destacadas.

function hacerPostulacionDestacada() {

    let idOferta = this.getAttribute("data-id");

    let respuesta = sistema.postularsePorId(sistema.usuarioLogueado, idOferta);

    document.querySelector("#pOfertasDestacadas").innerHTML = respuesta;

```

```

    mostrarOfertasDestacadas();

}

// Inicializa la pantalla de ofertas laborales y aplica el filtro por defecto.

function initOfertasPostulante(){

    initPostulante();

    let btnVolverMenuPostulante1 =
document.querySelector("#btnVolverMenuPostulante1");

    let btnAplicarFiltroOfertas = document.querySelector("#btnAplicarFiltroOfertas");

    // esto es para que por defecto muestre primero el area postulante

    mostrarOfertasPostulante("area");

    btnAplicarFiltroOfertas.addEventListener("click", aplicarFiltroOfertas);

    btnVolverMenuPostulante1.addEventListener("click", volverMenuPostulante);

}

// Lee el filtro seleccionado y actualiza el listado de ofertas.

function aplicarFiltroOfertas(){

    let filtro = document.querySelector("#slcFiltroOfertas").value;

```

```

    mostrarOfertasPostulante(filtro);

}

// Muestra las ofertas disponibles para el postulante según el filtro recibido.

function mostrarOfertasPostulante(filtro){

    let listadoOfertas = document.querySelector("#tbodyListadoOfertasPostulante");

    listadoOfertas.innerHTML = "";

    let ofertasTabla = sistema.obtenerOfertasParaPostulante(sistema.usuarioLogueado,
    filtro);

    for (let i = 0; i < ofertasTabla.length; i++) {

        let ofertaActual = ofertasTabla[i];

        listadoOfertas.innerHTML += `

        <tr>

            <td>${ofertaActual.getId()}</td>

            <td>${ofertaActual.titulo}</td>

            <td>${ofertaActual.empresa}</td>

            <td>${ofertaActual.nivel}</td>

```

```

<td>${ofertaActual.area}</td>

<td>${ofertaActual.destacada ? "★" : "-"}</td>

<td>${ofertaActual.getEstado()}</td>

<td>

    <button class="btnPostularme" data-id="${ofertaActual.getId()}">

        Postularme

    </button>

</td>

</tr>`;
}

```

```

let btnsPostularme = document.querySelectorAll(".btnPostularme");

for(let i = 0; i < btnsPostularme.length; i++){

    btnsPostularme[i].addEventListener("click", hacerPostulacion);

}

}

```

// Procesa la postulación desde el listado general de ofertas.

```

function hacerPostulacion() {

    let idOferta = this.getAttribute("data-id");

```

```
let respuesta = sistema.postularsePorId(sistema.usuarioLogueado, idOferta);

document.querySelector("#pOfertasPostulante").innerHTML = respuesta;


aplicarFiltroOfertas();

}
```

```
function initRegistro() {

    let btnRegistrar = document.querySelector("#btnRegistrar");

    let btnVolverLogin = document.querySelector("#btnVolverLogin");


    btnRegistrar.addEventListener("click", registrarPostulante);

    btnVolverLogin.addEventListener("click", irLogin);

}
```

```
// Lee los datos del formulario y se los pasa al Sistema.
```

```

// La validación real y el push al array se hacen en sistema.js.

function registrarPostulante() {

    let usuario = document.querySelector("#txtUsuario").value.trim().toLowerCase();

    let password = document.querySelector("#txtPassword").value;

    let nombre = document.querySelector("#txtNombre").value.trim();

    let experiencia = document.querySelector("#slcExperiencia").value;

    let area = document.querySelector("#slcArea").value;

    let pResultado = document.querySelector("#pResultado");

    let respuesta = sistema.registrarPostulante(

        usuario,

        password,

        nombre,

        experiencia,

        area

    );

    pResultado.innerHTML = respuesta;

}

/* vuelve al template de login con los parametros, de la funcion irA */

```

```
function irLogin() {
```

```
    irA("view-login", initLogin);
```

```
}
```

```
// funcion para navegar entre template
```

```
function irA(idTemplate, funcionInicio) {
```

```
    let contenedor = document.querySelector("#contenedorPrincipal");
```

```
    let template = document.querySelector("#" + idTemplate);
```

```
    contenedor.innerHTML = template.innerHTML;
```

```
    if (funcionInicio !== undefined) { //puede tomarmelo como falso cuando es 0 o no existe.
```

```
        funcionInicio();
```

```
}  
  
}
```

```
// para buscar en un array
```

```
/*
```

```
lista: array donde buscar.
```

```
atributo: propiedad a comparar.
```

```
valor: dato que quiero encontrar.
```

```
Devuelve la posición o -1 si no existe.
```

```
*/
```

```
function findCasero(lista, atributo, valor) {
```

```
    for (let i = 0; i < lista.length; i++) {
```

```
        let elementoActual = lista[i];
```

```
        if (elementoActual[atributo] === valor) {
```

```

        return i;

    }

}

return -1;

}

//hay que llamarla y buscar con esto

//me sirve SOLO para atributos PUBLICOSSSSS

/* este es mi find casero para los id

*/

function findCaseroID(lista, valor) {

    for (let i = 0; i < lista.length; i++) {

        let elementoActual = lista[i];

        if (elementoActual.getId() === valor) {

            return i;

```

```

    }

}

return -1;

}

// Verifica si la contraseña cumple con los requisitos solicitados.

function validarPassword(password) {

    let tieneMayus = false;

    let tieneMinus = false;

    let contadorNumPas = 0;

    for (let i = 0; i < password.length; i++) {

        let letra = password.charAt(i);

        if (!isNaN(Number(letra))) {

            contadorNumPas++;

        }
    }

```

```

    if (letra.charCodeAt(0) >= 65 && letra.charCodeAt(0) <= 90) {

        tieneMayus = true;

    }

    if (letra.charCodeAt(0) >= 97 && letra.charCodeAt(0) <= 122) {

        tieneMinus = true;

    }

}

return {

    tieneMayus,

    tieneMinus,

    contadorNumPas

};

}

// Valida los datos ingresados para registrar un postulante.

function validarRegistroPostulante(usuario, password, nombre, experiencia, area,
sistema) {

    let respuesta = "";

```

```
let resultadoContra = validarPassword(password);
```

```
if (usuario === "") {
```

```
    respuesta += "el usuario no debe estar vacio<br>";
```

```
}
```

```
if (usuario.length < 5) {
```

```
    respuesta += "el usuario debe tener al menos 5 caracteres<br>";
```

```
}
```

```
if (password === "") {
```

```
    respuesta += "la contraseña no debe estar vacia<br>";
```

```
}
```

```
if (password.length < 5) {
```

```
    respuesta += "la contraseña debe tener al menos 5 caracteres<br>";
```

```
}
```

```
if (nombre.length < 5) {
```

```
    respuesta += "el nombre debe tener al menos 5 caracteres<br>";
```

```
}
```

```
if (experiencia === "") {  
  
    respuesta += "seleccione una opcion de experiencia<br>";  
  
}
```

```
if (area === "") {  
  
    respuesta += "seleccione una opcion de area<br>";  
  
}
```

```
let usuarioBuscado = usuario.toLowerCase();
```

```
for(let i = 0; i < sistema.postulantes.length; i++){  
  
    if(sistema.postulantes[i].usuario.toLowerCase() === usuarioBuscado){  
  
        respuesta += "el usuario ya existe<br>";  
  
    }  
  
}
```

```
if (resultadoContra.contadorNumPas === 0) {  
  
    respuesta += "la contraseña debe tener al menos un numero<br>";  
  
}
```

```

if (resultadoContra.tieneMayus === false) {

    respuesta += "la contraseña debe tener al menos una mayuscula<br>";

}

if (resultadoContra.tieneMinus === false) {

    respuesta += "la contraseña debe tener al menos una minuscula<br>";

}

return respuesta;

}

// Verifica si una oferta puede ser mostrada y utilizada por un postulante.

function validarOfertaParaPostulante(postulante, oferta) {

    if (

        oferta.getEstado() === "Activa" &&

        sistema.expCompatible(postulante, oferta) &&

        !sistema.yaSePostulo(postulante, oferta) &&

        sistema.contarPostulacionesOferta(oferta) < oferta.limitePostulaciones

    ) {

        return true;
    }
}

```

```

    }

    return false;
}

// Valida los datos necesarios para crear o editar una oferta laboral.

function validarOferta(titulo, empresa, descripcion, nivel, area, limitePostulaciones,
cantidadVacantes, destacada) {

    if (

        titulo === "" ||

        empresa === "" ||

        descripcion === "" ||

        nivel === "" ||

        area === "" ||

        limitePostulaciones === "" ||

        cantidadVacantes === "" ||

        destacada === ""

    ) {

        return "Todos los campos son obligatorios";

    }

```

```

if (Number(limitePostulaciones) < Number(cantidadVacantes)) {

    return "El límite de postulaciones debe ser mayor o igual a la cantidad de vacantes";

}

if (Number(limitePostulaciones) <= 0 || Number(cantidadVacantes) <= 0) {

    return "El límite de postulaciones y la cantidad de vacantes deben ser mayores a 0";

}

return "";

}

//MAIN

irA("view-login", initLogin);

let sistema = new Sistema();

//HTML

```

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <link rel="stylesheet" href="styles.css">
```

```
  <title>Cartelera Laboral</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Search work</h1>
```

```
<div id="contenedorPrincipal"></div>
```

```
<!-- LOGIN -->
```

```
<template id="view-login">
```

```
  <section>
```

```
    <h2>Iniciar sesión</h2>
```

```
    <label for="txtUsuarioLogin">Usuario</label>
```

```
    <input type="text" id="txtUsuarioLogin">
```

<label for="txtPasswordLogin">Contraseña</label>

<input type="password" id="txtPasswordLogin">

<button id="btnLogin">Ingresar</button>

<button id="btnIrRegistro">Registrarme</button>

<p id="pLogin"></p>

</section>

</template>

<template id="view-registro">

<section >

<h2>Registro de Postulante</h2>

<label for="txtUsuario">Usuario</label>

<input type="text" id="txtUsuario">

<label for="txtPassword">Contraseña</label>

<input type="password" id="txtPassword">

<label for="txtNombre">Nombre completo</label>

<input type="text" id="txtNombre">

<label for="slcExperiencia">Experiencia</label>

<select id="slcExperiencia">

<option value="">Seleccione...</option>

<option value="Junior">Junior</option>

<option value="Semi-Senior">Semi-Senior</option>

<option value="Senior">Senior</option>

</select>

<label for="slcArea">Área</label>

<select id="slcArea">

<option value="">Seleccione...</option>

<option value="Tecnología">Tecnología</option>

<option value="Diseño">Diseño</option>

<option value="Marketing">Marketing</option>

<option value="Administración">Administración</option>

<option value="Otros">Otros</option>

</select>

<button id="btnRegistrar">Registrarse</button>

<button id="btnVolverLogin">Volver al login</button>

<p id="pResultado"></p>

</section>

</template>

<template id="view-ofertas-postulante">

<nav class="navbar">

<div class="infoMenu">

<h3>POSTULANTE</h3>

<p>Portal laboral</p>

</div>

Ofertas laborales

Mis postulaciones

Ofertas destacadas

Cerrar sesión

</nav>

```
<section>
```

```
  <h2>Ofertas laborales activas</h2>
```

```
  <label for="slcFiltroOfertas">Filtro</label>
```

```
  <select id="slcFiltroOfertas">
```

```
    <option value="todas">Ver todas </option>
```

```
    <option value="area" selected>Ver solo mi area</option>
```

```
  </select>
```

```
  <button id="btnAplicarFiltroOfertas">Aplicar filtro</button>
```

```
  <table>
```

```
    <thead>
```

```
      <tr>
```

```
        <th>ID</th>
```

```
        <th>titulo</th>
```

```
        <th>empresa</th>
```

```
        <th>nivel</th>
```

```
        <th>area</th>
```

```
        <th>destacada</th>
```

```
        <th>Estado</th>
```

```
        <th>Acciones</th>
```

```

        </tr>

    </thead>

    <tbody id="tbodyListadoOfertasPostulante"></tbody>

</table>

<p id="pOfertasPostulante"></p>

<button id="btnVolverMenuPostulante1">Volver</button>

</section>

</template>

<template id="view-mis-postulaciones">

    <nav class="navbar">

        <div class="infoMenu">

            <h3>POSTULANTE</h3>

            <p>Portal laboral</p>

        </div>

        <a href="#" id="btnVerOfertas">Ofertas laborales</a>

        <a href="#" id="btnVerMisPostulaciones">Mis postulaciones</a>

```

`<a href="#" id="btnVerOfertasDestacadas">Ofertas destacadas</a>`

`<a href="#" id="btnCerrarSesionPostulante">Cerrar sesión</a>`

`</nav>`

`<section>`

`<h2>Mis postulaciones</h2>`

`<p id="pMisPostulaciones"></p>`

`<table>`

`<thead>`

`<tr>`

`<th>ID</th>`

`<th>titulo</th>`

`<th>empresa</th>`

`<th>nivel</th>`

`<th>area</th>`

`<th>destacada</th>`

`<th>Estado</th>`

</tr>

</thead>

<tbody id="tbodyListadoMisPos"></tbody>

</table>

<button id="btnVolverMenuPostulante2">Volver</button>

</section>

</template>

<template id="view-ofertas-destacadas">

<nav class="navbar">

<div class="infoMenu">

<h3>POSTULANTE</h3>

<p>Portal laboral</p>

</div>

Ofertas laborales

Mis postulaciones

Ofertas destacadas

Cerrar sesión

</nav>

<section id="pantallaOfertasDestacadas">

<h2>Ofertas destacadas</h2>

<p id="pOfertasDestacadas"></p>

<table>

<thead>

<tr>

<th>titulo</th>

<th>empresa</th>

<th>nivel</th>

<th>area</th>

<th>destacada</th>

<th>Descripción</th>

<th>Estado</th>

<th>Acciones</th>

</tr>

</thead>

<tbody id="tbodyListadoDestacadas"></tbody>

</table>

<button id="btnVolverMenuPostulante3">Volver</button>

</section>

</template>

<template id="view-admin">

<nav class="navbar">

Crear oferta laboral

Listado de ofertas

Postulaciones pendientes

Estadísticas

Cerrar sesión

</nav>

```

<section id="pantallaAdmin">

    <h2>Menú Administrador</h2>

    <p id="pBienvenidaAdmin"></p>

</section>

</template>

<template id="view-crear-oferta">

    <nav class="navbar">

        <div class="infoMenu">

            <h3>ADMIN</h3>

            <p>Panel de control</p>

            </div>

            <a href="#" id="btnCrearOferta">Crear oferta laboral</a>

            <a href="#" id="btnVerListadoOfertasAdmin">Listado de ofertas</a>

            <a href="#" id="btnVerPostulacionesPendientes">Postulaciones pendientes</a>

            <a href="#" id="btnVerEstadisticasAdmin">Estadísticas</a>

            <a href="#" id="btnCerrarSesionAdmin">Cerrar sesión</a>

        </nav>

```

<section id="pantallaCrearOferta">

<h2>Crear oferta laboral</h2>

<label for="txtTituloOferta">Título de la oferta</label>

<input type="text" id="txtTituloOferta">

<label for="txtEmpresaOferta">Empresa</label>

<input type="text" id="txtEmpresaOferta">

<label for="txtDescripcionOferta">Descripción</label>

<textarea id="txtDescripcionOferta"></textarea>

<label for="slcNivelOferta">Nivel requerido</label>

<select id="slcNivelOferta">

<option value="">Selecione...</option>

<option value="Junior">Junior</option>

<option value="Semi-Senior">Semi-Senior</option>

<option value="Senior">Senior</option>

</select>

<label for="slcAreaOferta">Área</label>

<select id="slcAreaOferta">

<option value="">Selecione...</option>

<option value="Tecnología">Tecnología</option>

<option value="Diseño">Diseño</option>

<option value="Marketing">Marketing</option>

<option value="Administración">Administración</option>

<option value="Otros">Otros</option>

</select>

<label for="txtLimitePostulaciones">Límite de postulaciones</label>

<input type="number" id="txtLimitePostulaciones">

<label for="txtCantidadVacantes">Cantidad de vacantes</label>

<input type="number" id="txtCantidadVacantes">

<label for="slcOfertaDestacada">Oferta destacada</label>

<select id="slcOfertaDestacada">

<option value="">Selecione...</option>

<option value="si">Sí</option>

<option value="no">No</option>

</select>

<button id="btnGuardarOferta">Guardar oferta</button>

<button id="btnVolverMenuAdmin1">Volver</button>

<p id="pCrearOferta"></p>

</section>

</template>

<template id="view-table-ofertas-admin">

<nav class="navbar">

<div class="infoMenu">

<h3>ADMIN</h3>

<p>Panel de control</p>

</div>

Crear oferta laboral

Listado de ofertas

Postulaciones pendientes

Estadísticas

Cerrar sesión

</nav>

<section id="pantallaListadoOfertasAdmin">

<h2>Listado de ofertas</h2>

<table>

<thead>

<tr>

<th>ID</th>

<th>titulo</th>

<th>empresa</th>

<th>nivel</th>

<th>area</th>

<th>destacada</th>

<th>Estado</th>

<th>Acciones</th>

</tr>

</thead>

<tbody id="tbodyListadoOfertasAdmin"></tbody>

</table>

<p id="pListadoOfertasAdmin"></p>

<button id="btnVolverMenuAdmin2">Volver</button>

</section>

</template>

<!-- CORREGI EL TEMPLATE QUE DE EDITAR OFERTAS , SE ENCONTRABA DENTRO DEL TEMPLATE DE LISTADO DE OFERTAS Y EN TEORIA SON VISTAS SEPARADAS , CHECK CON NAZA POR LAS DUDAS -->

<template id="view-editar-oferta">

<nav class="navbar">

<div class="infoMenu">

<h3>ADMIN</h3>

<p>Panel de control</p>

</div>

Crear oferta laboral

Listado de ofertas

Postulaciones pendientes

Estadísticas

Cerrar sesión

</nav>

<section>

<h3>Editar oferta</h3>

<input type="hidden" id="txtIdOfertaEditando">

<label>Título</label>

<input type="text" id="txtEditarTituloOferta">

<label>Empresa</label>

<input type="text" id="txtEditarEmpresaOferta">

<label>Descripción</label>

<textarea id="txtEditarDescripcionOferta"></textarea>

<label>Nivel</label>

<select id="slcEditarNivelOferta">

<option value="Junior">Junior</option>

<option value="Semi-Senior">Semi-Senior</option>

<option value="Senior">Senior</option>

</select>

<label>Área</label>

<select id="slcEditarAreaOferta">

<option value="Tecnología">Tecnología</option>

<option value="Diseño">Diseño</option>

<option value="Marketing">Marketing</option>

<option value="Administración">Administración</option>

<option value="Otros">Otros</option>

</select>

<label>Límite postulaciones</label>

<input type="number" id="txtEditarLimitePostulaciones">

<label>Cantidad vacantes</label>

<input type="number" id="txtEditarCantidadVacantes">

<label>Destacada</label>

<select id="slcEditarOfertaDestacada">

<option value="si">Sí</option>

<option value="no">No</option>

</select>

<button id="btnGuardarCambiosOferta">Guardar cambios</button>

<button id="btnCancelarEditarOferta">Cancelar</button>

<p id="pEditarOferta"></p>

</section>

</template>

<template id="view-postulaciones-pendientes">

<nav class="navbar">

<div class="infoMenu">

<h3>ADMIN</h3>

<p>Panel de control</p>

</div>

Crear oferta laboral

Listado de ofertas

Postulaciones pendientes

Estadísticas

```
<a href="#" id="btnCerrarSesionAdmin">Cerrar sesión</a>
```

```
</nav>
```

```
<section id="pantallaPostulacionesPendientes">
```

```
<table>
```

```
<thead>
```

```
<tr>
```

```
<th>ID</th>
```

```
<th>Postulante</th>
```

```
<th>Experiencia</th>
```

```
<th>Área de interes</th>
```

```
<th>Oferta</th>
```

```
<th>Empresa</th>
```

```
<th>Nivel requerido</th>
```

```
<th>Acciones</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody id="tbodyPostulacionesPendientes">
```

```
</tbody>
```

```
</table>
```

<p id="pPostulacionesPendientes"></p>

<button id="btnVolverMenuAdmin3">Volver</button>

</section>

</template>

<template id="view-estadisticas-admin">

<nav class="navbar">

<div class="infoMenu">

<h3>ADMIN</h3>

<p>Panel de control</p>

</div>

Crear oferta laboral

Listado de ofertas

Postulaciones pendientes

Estadísticas

Cerrar sesión

</nav>

<section id="pantallaEstadisticasAdmin">

<h2>Estadísticas del sistema</h2>

<label for="txtBuscarOfertaEstadistica">Buscar oferta por título</label>

<input type="text" id="txtBuscarOfertaEstadistica">

<button id="btnBuscarOfertaEstadistica">Buscar</button>

<h3>Postulaciones por oferta</h3>

<table>

<thead>

<tr>

<th>Título de la oferta</th>

<th>Pendientes</th>

<th>Aceptadas</th>

<th>Rechazadas</th>

<th>Total</th>

</tr>

</thead>

<tbody id="tbodyPostulacionesPorOferta">

</tbody>

</table>

<h3>Total de ofertas por estado</h3>

<table>

<thead>

<tr>

<th>activas</th>

<th>inactivas</th>

<th>cerradas</th>

</tr>

</thead>

<tbody id="tbodyTotalOfertasPorEstado">

</tbody>

</table>

<h3>Resumen general</h3>

```
<table>
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Indicador</th>
```

```
      <th>Resultado</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody id="tbodyResumenGeneral">
```

```
  </tbody>
```

```
</table>
```

```
<button id="btnVolverMenuAdmin4">Volver</button>
```

```
</section>
```

```
</template>
```

```
<script src="js/class/Admin.js"></script>
```

```
<script src="js/class/Postulante.js"></script>
```

<script src="js/class/OfertaLaboral.js"></script>

<script src="js/class/Postulacion.js"></script>

<script src="js/utills/search.js"></script>

<script src="js/utills/validaciones.js"></script>

<script src="js/class/sistema.js"></script>

<script src="js/utills/navegacion.js"></script>

<script src="js/page/ViewLogin.js"></script>

<script src="js/page/ViewRegistro.js"></script>

<script src="js/page/ViewAdmin.js"></script>

<script src="js/page/ViewPostulante.js"></script>

<script src="js/main.js"></script>

</body>

</html>